# flashjet

GPU-native jet clustering with the full merge history

Chirayu Gupta

ML4Jets 2026 · Vienna

September 2026

Benchmarks: **ATLAS Open Data** (CC0). Closures: analytic predictions on toy showers.

# Outline

# Jet clustering, and flashjet

# Reconstruction is moving to accelerators — clustering has not

**Reconstruction is going heterogeneous**

- CMS runs GPUs in the HLT since Run 3
- $\sim$**30 %** of HLT reconstruction offloaded
- $\rightarrow$ $\sim$**25 %** less total HLT time

pixel local reco · pixel tracks + vertices · calorimeter local reco

**Ported so far**

- tracking · vertexing · calorimetry

**not jet clustering**

## Why it is still on the CPU

- sequential recombination is **IRC safe** — why it survived
- but each merge **depends on the previous one**
- that data dependence does not map onto a GPU in the obvious way

## Where it hurts

- reclustering **inside a training loop**
- scanning the **jet radius**
- **substructure** on demand

each one pays a host $\leftrightarrow$ device transfer, over and over

# flashjet

**An open-source GPU jet clusterer**

- $k_t$, Cambridge–Aachen, anti-$k_t$
- custom **Triton** kernels

**Why Triton**

- the GPU language of the PyTorch ecosystem
- **compiles and autotunes** for the hardware at hand
- **no separate CUDA build**, no device code to maintain

**Execution**

- one program clusters **one event on chip**
- the batch goes out in **one fused launch**
- jets, histories, substructure $\rightarrow$ **tensors**

## What makes it tractable

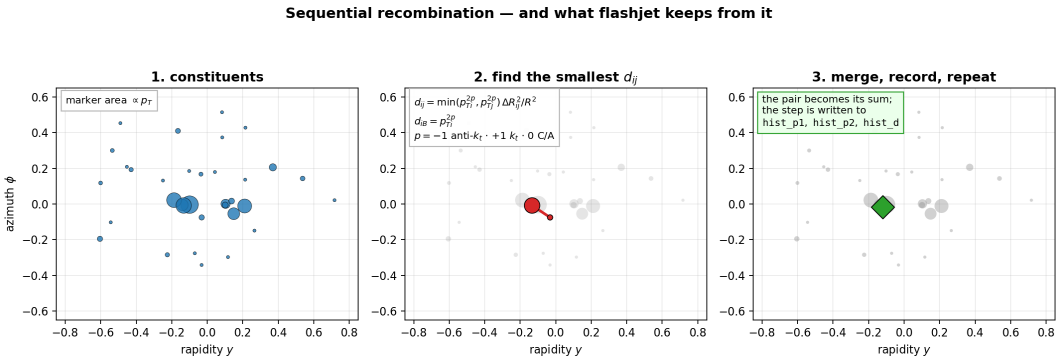
Cacciari–Salam NN lemma: the global $d_{ij}$ minimum sits at some particle's **geometric** nearest neighbour.

- each slot carries its own NN
- a merge invalidates only $O(1)$ rows
- $\Rightarrow O(N^2)$**, not** $O(N^3)$

## Engineering

- bandwidth-bound $\rightarrow$ NN update and **next argmin** fused into one sweep
- **branchless** pair-vs-beam (masked stores)
- autotuned **once per GPU model**, cached $\rightarrow$ **reproducible**

# How sequential recombination works




Repeat until nothing is left: if the smallest distance is a $d_{ij}$, merge that pair; if it is a $d_{iB}$, call that a jet and remove it. One exponent $p$ selects the algorithm — the *same* kernel serves all three.

# One exponent, three algorithms

The generalised-$k_t$ distance measure:

$$d_{ij} = \min\left(p_{Ti}^{2p}, p_{Tj}^{2p}\right) \frac{\Delta R_{ij}^2}{R^2}, \qquad \Delta R_{ij}^2 = (y_i - y_j)^2 + (\phi_i - \phi_j)^2$$

$$d_{iB} = p_{Ti}^{2p}$$

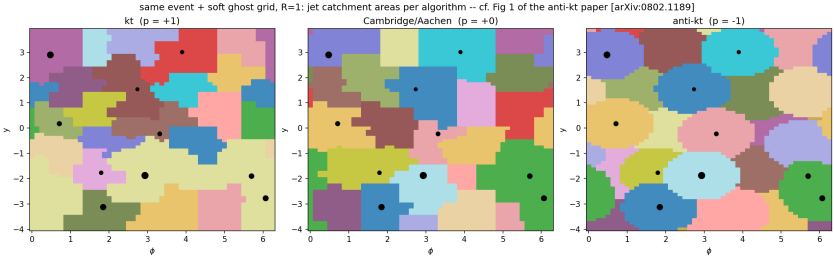
Merge the pair with the smallest $d_{ij}$; if the smallest distance is a $d_{iB}$, call $i$ a jet and remove it.

**Only the exponent $p$ changes:**

| $p$ | algorithm | merges first | the tree is ordered by |
|---|---|---|---|
| $-1$ | anti-$k_t$ | around the **hardest** particles | — (gives the jets) |
| $0$ | Cambridge–Aachen | the **closest** pair | **angle** |
| $+1$ | $k_t$ | the **softest** pair | **relative** $p_T$ |

The ordering is **what the substructure reads**: C/A is the tree for **grooming** and the **Lund plane**, $k_t$ for **exclusive subjets**.

# anti-$k_t$ gives rigid, circular jets




Jet catchment areas for the three algorithms, from flashjet's own clustering. $k_t$ and C/A give ragged, fluctuating boundaries; **anti-$k_t$ gives circles of area $\pi R^2$** around each hard particle — the defining result of the algorithm, recovered here without reference to FastJet.

# It returns the tree, not just the jets

Clustering a batch returns **two** things — the jets, and the record of how they were built.

| field | shape | what it holds |
|---|---|---|
| `jet_idx` | $(B, N)$ | which jet each particle ended up in |
| `n_jets` | $(B,)$ | how many jets were found |
| `hist_p1`, `hist_p2` | $(B, N)$ | **which pair merged**, at each step |
| `hist_child` | $(B, N)$ | what they merged *into* |
| `hist_d` | $(B, N)$ | the distance $d_{ij}$ at that step |

## Why the second half matters

The bottom three rows are the **complete binary merge tree**. Groomed mass, $z_g$, $R_g$, Lund coordinates and exclusive subjets are all **coordinates on that tree** — so they become array reads, with **no second clustering pass**.

# Using it

```
import flashjet

out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
```

| argument | type | |
|---|---|---|
| p4 | $(B, N, 4)$ tensor | $(p_x, p_y, p_z, E)$; CPU or CUDA |
| mask | $(B, N)$ bool | marks real constituents in a padded batch |
| R | float | jet radius |
| algorithm | str | "antikt" \| "kt" \| "cambridge" |
| p | float | generalised-$k_t$ exponent (overrides algorithm) |

```
out.n_jets                                    # (B,)     jets found
out.jets_p4(p4)                               # (B,J,4)  jet four-momenta
out.splitting_scales()                        #          sqrt(d12), sqrt(d23), ...
out.exclusive_jets(n_jets=3)                  # F1       exclusive kt subjets
out.groomed_jets(p4, R, z_cut=0.1, beta=0.0)  # F2       soft drop / mMDT
out.lund_coordinates(p4, R)                   # F3       primary Lund plane
```

The same call runs unchanged on CPU and GPU — the backend is chosen from the problem size, not by the caller.

# What is implemented

**Clustering**

- generalised-$k_t$: anti-$k_t$, $k_t$, C/A
- batched, ragged input via a mask
- three backends, auto-dispatched by size
- merge history recorded in-kernel
- jet regime *and* event regime

**Substructure** — reads of the history

- **F1** exclusive $k_t$ subjets
- **F2** soft drop / mMDT / mass-drop
- **F3** primary Lund coordinates

All three are pure-torch post-reads: no kernel changes, CPU/GPU identical, negligible cost.

# F1 — exclusive subjets from the $k_t$ history

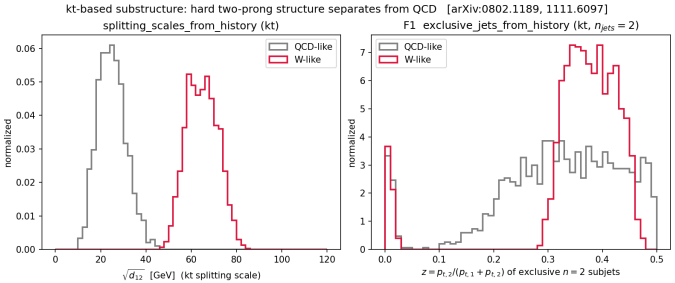
**What it does.** Undo the last merges of the sequence to expose exactly $n$ subjets — or stop at a $d_{cut}$.

**How.** The history is already a binary tree. Walk back up it $n-1$ steps; no reclustering.

**Why $k_t$.** $k_t$ merges the softest pair first, so the *last* merges are the hard prongs — exactly what a 2-prong decay leaves behind.

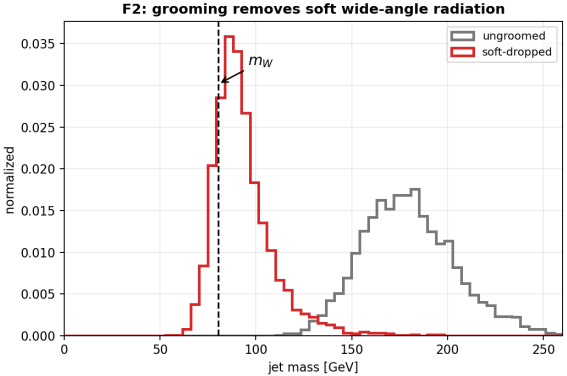
The splitting scales come straight from `hist_d`:

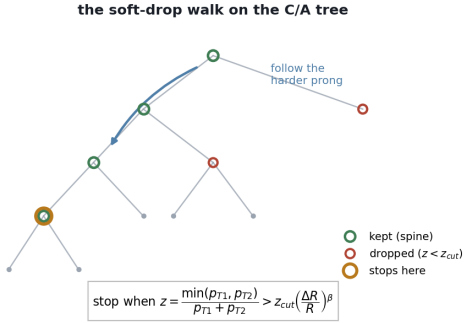
$$\sqrt{d_{12}},\ \sqrt{d_{23}},\ \ldots \quad \text{with} \quad d_{ij} = \min(p_{Ti}^2, p_{Tj}^2)\,\Delta R_{ij}^2/R^2$$




Toy W-like (2-prong) vs QCD-like fat jets. **Left:** the $k_t$ splitting scale $\sqrt{d_{12}}$ separates cleanly. **Right:** the momentum balance $z$ of the two exclusive subjets — the W sits near $z \sim 0.4$, QCD spreads to small $z$.

# F2 — soft drop / mMDT grooming from the C/A history

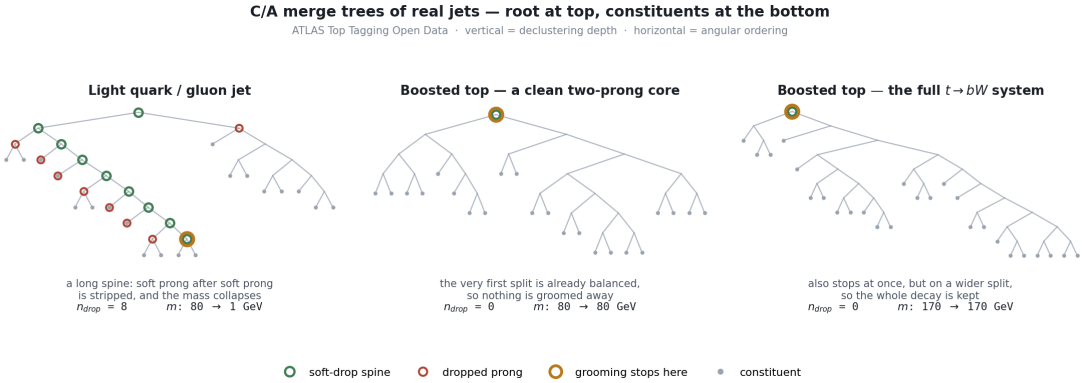



Walk down the C/A tree along the harder branch, discarding the softer prong until the momentum-sharing condition is met. $\beta = 0$ recovers mMDT.

The walk is $O(\log_2 n)$ on the stored tree — it never touches the constituents again, so grooming is essentially free once clustered.

# What the tree looks like on real jets

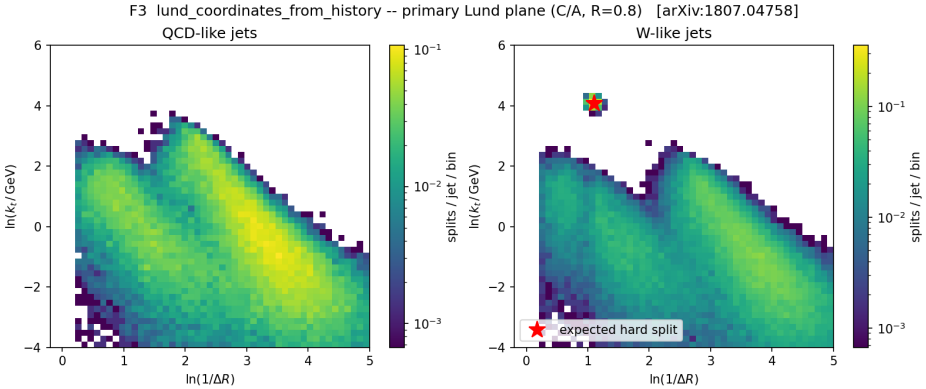



**A jet without a decay has nothing to stop on.** The light-quark/gluon jet is a long *staircase*: soft prong after soft prong is stripped, and its mass collapses to nothing.

**A real decay stops at once.** Both top jets halt on their very first split, because it is already hard and balanced — so the mass survives.

ATLAS Open Data, C/A with soft drop ($z_{cut}=0.1$, $\beta=0$). **The number of drops is itself a discriminant** — and it is read straight off the stored tree, at no extra cost.

# F3 — primary Lund coordinates from the C/A history




**What.** Every primary splitting emits $z = \frac{p_{T2}}{p_{T1}+p_{T2}}$, $\Delta R$, $k_t = p_{T2}\Delta R$, and the plane coordinates $\left(\ln\frac{1}{\Delta R},\ \ln k_t\right)$.

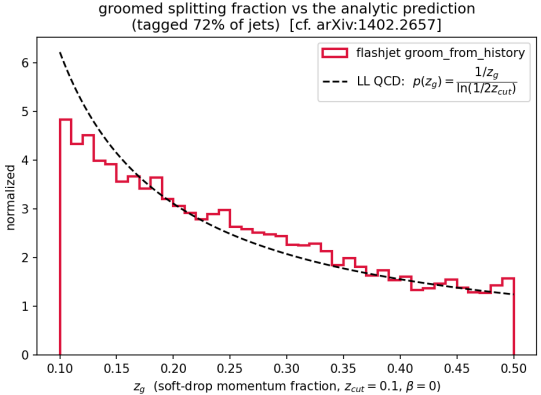
**How.** One pass down the primary branch — each node is one point in the plane.

**Check.** The $d$-channel ties *exactly* to the splitting scales computed independently.

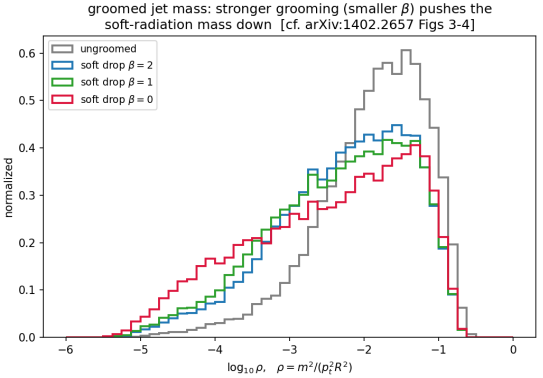
QCD fills the soft-collinear region smoothly; the W-like sample adds a **localised hard splitting** exactly where the decay is predicted to sit (star).

# Correctness

# The history is right: closures against analytic predictions



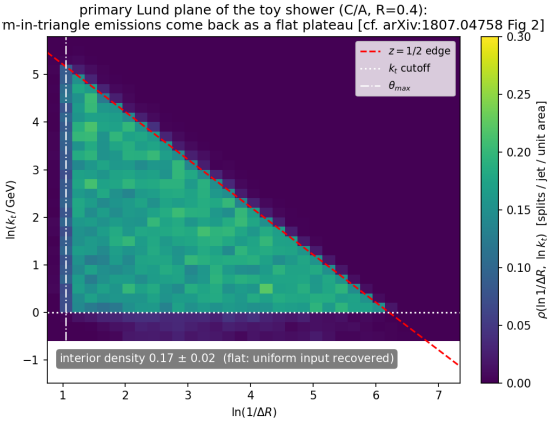

Soft-drop $z_g$ follows the $1/z$ form predicted at leading-log, recovered from the C/A history.



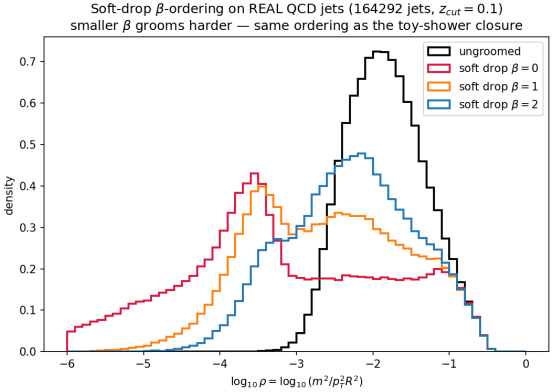

Groomed-mass ordering in $\beta$: larger $\beta$ grooms less, so the mass peak moves up.

Toy-shower input, so the *prediction* is unambiguous — these test the decoders, not an event generator.

# Clustering geometry




The Lund plane is populated **uniformly inside the kinematic triangle** — the leading-log expectation.




The soft-drop $\beta$-family orders as predicted: larger $\beta$ grooms less.

Independent NumPy tree-walks pin every decoder, and the $d$-channel of the Lund output ties *exactly* to the splitting scales computed separately.

# Same jets as FastJet — across the whole grid

**3 algorithms** $\times$ **5 radii** $\times$ **5 multiplicity bins** $\times$ **2 samples**, 20 000 jets per bin, on ATLAS Open Data.

| **check** | **result** |
|---|---|
| number of jets found | **100.000 %** at every one of 150 points |
| leading-jet $p_T$ within $10^{-4}$ | 1.0000 at 148/150 points |
| median relative difference | $\sim 7 \times 10^{-8}$ |

# Same jets as the experiment

A stronger test than agreeing with FastJet.

ATLAS publishes an open dataset carrying both the **calorimeter clusters** and **its own reconstructed jets**. Cluster the ∼600 clusters per event ourselves, and compare:

| | |
|---|---|
| $n_{\text{jets}}$ vs ATLAS | **100.0 %** match |
| mean jets/event | 10.96 vs 10.96 |

This closes the algorithm against **the experiment's own published output** — not against another implementation of the same algorithm.

599.3 clusters/event: the event regime, where the problem is $O(N^2)$.

# Speed

# The benchmark

**Data** — ATLAS Open Data, freely presentable

| sample | jets | $\langle n_{\text{const}} \rangle$ |
|---|---|---|
| boosted top | 30 000 | 59.2 |
| QCD (light $q/g$) | 30 000 | 52.4 |

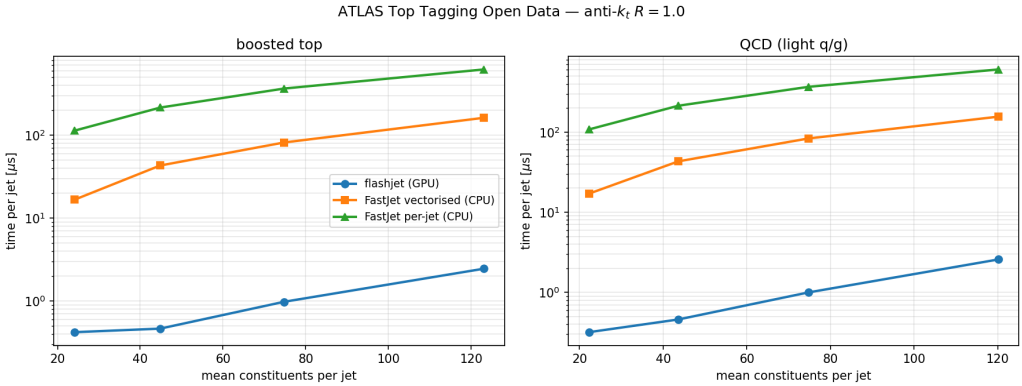
Large-$R$ jets with calorimeter-cluster constituents. The grid is **3 algorithms** $\times$ **5 radii** $\times$ **5 multiplicity bins**, 20 000 jets per bin.

**Method**

1. extract constituents once, save;
2. cluster on GPU, time it, save the *same* events;
3. cluster those saved events with FastJet;
4. **check they agree** — else the timing is meaningless.

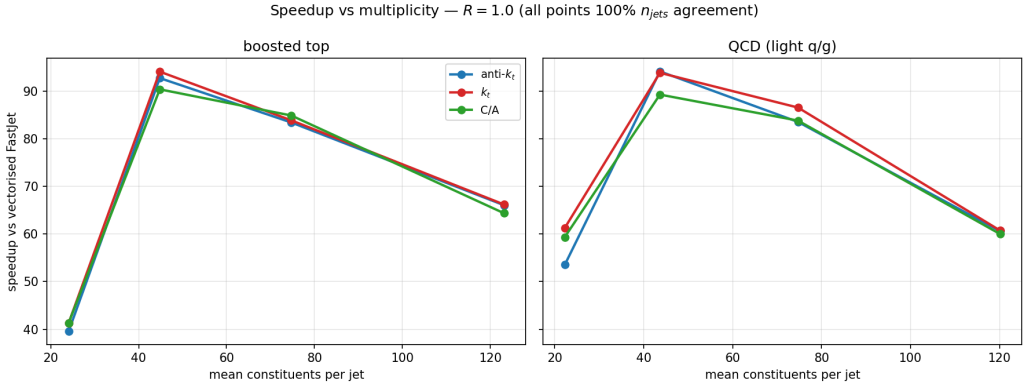
Both clusterers read the **same saved file**. Speedups are quoted against the **vectorised** FastJet interface — the faster, fairer baseline.

# Time per jet



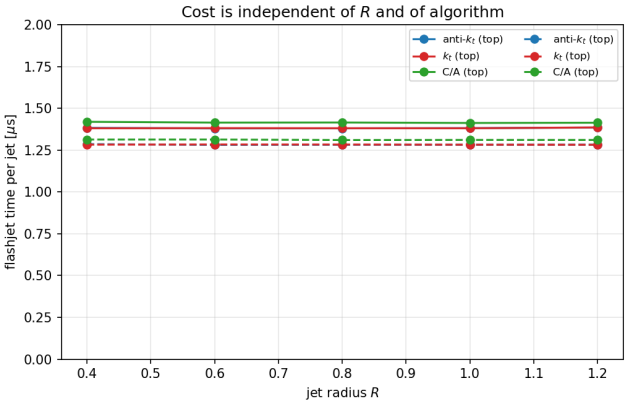

**Log scale** — each gridline is $10\times$. flashjet stays nearly flat while both FastJet interfaces climb steeply with multiplicity: **0.35–2.7 $\mu$s per jet** against **17–155 $\mu$s** for vectorised FastJet, and **110–620 $\mu$s** per jet one at a time. Benchmarked on an NVIDIA **V100-class** GPU.

# The advantage grows with jet complexity




All three algorithms, both samples, $R = 1.0$. **39–99**× over vectorised FastJet across the grid (median **64**×), and every point has **100 %** $n_{\text{jets}}$ agreement. Busier jets → bigger win — the regime that matters for boosted-object physics.

# Cost is independent of $R$ — and of the algorithm




Across $R = 0.4 \to 1.2$: a **0.3 % spread** over a $3\times$ range in $R$.

$R$ changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti-$k_t$, $k_t$ and C/A agree to within $\sim$4 % at every multiplicity.

$k_t$ and C/A **cost the same as anti-$k_t$.**

# Summary

1. **Clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjets, soft drop, Lund coordinates — comes free as array reads.
2. **Gives the same jets as FastJet.** 150/150 grid points at 100 % $n_{\text{jets}}$ agreement, across anti-$k_t$/$k_t$/C-A, $R = 0.4$–$1.2$ and 24–123 constituents per jet.
3. **Reproduces the experiment's own reconstructed jets** exactly, at ∼600 clusters per event.
4. **39–99× faster** than vectorised FastJet, and the advantage *grows* with jet complexity. Cost is independent of $R$ and of algorithm.
5. Still a **prototype**, with meaningful headroom left.

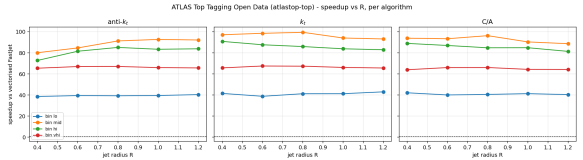
# Thank you

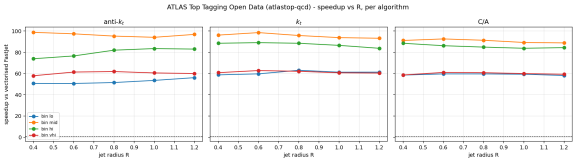
Questions?

Benchmarks: ATLAS Open Data (CC0).
Physics closures: analytic predictions on toy showers.

# Backup

# Backup — speedup vs $R$, per algorithm



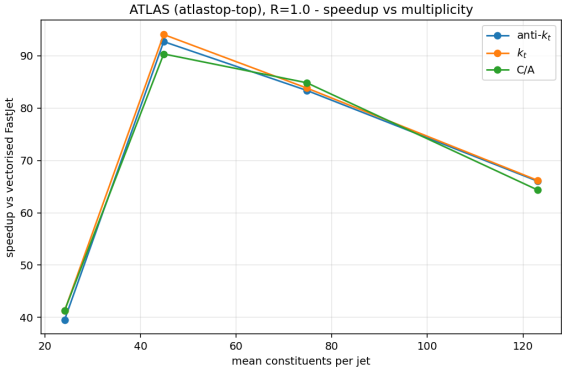




boosted top

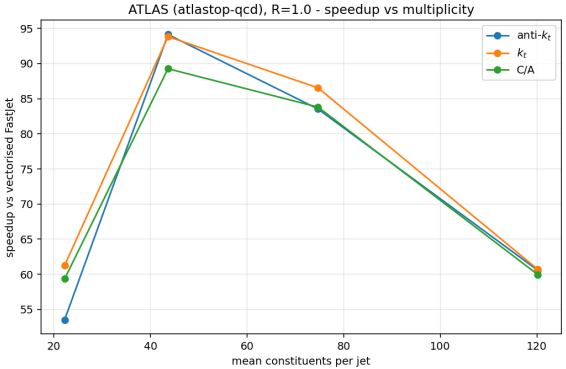
QCD (light $q/g$)

Flat in $R$ for all three algorithms; ordering is by multiplicity bin.

# Backup — speedup vs multiplicity




boosted top




QCD (light $q/g$)

# Backup — the numbers

| sample | alg | $\langle n \rangle$ | flashjet [$\mu$s/jet] | FastJet vec. [$\mu$s/jet] | speedup |
|---|---|---|---|---|---|
| top | anti-$k_t$ | 24.1 | 0.427 | 16.5 | $39\times$ |
| top | anti-$k_t$ | 44.8 | 0.464 | 43.0 | $\mathbf{93}\times$ |
| top | C/A | 74.7 | 1.000 | 84.8 | $85\times$ |
| top | C/A | 123.1 | 2.565 | 165.1 | $64\times$ |
| qcd | anti-$k_t$ | 22.2 | 0.346 | 17.5 | $51\times$ |
| qcd | anti-$k_t$ | 43.6 | 0.463 | 45.7 | $\mathbf{99}\times$ |
| qcd | anti-$k_t$ | 74.7 | 1.215 | 89.8 | $74\times$ |
| qcd | anti-$k_t$ | 120.2 | 2.673 | 154.9 | $58\times$ |

**39–99**$\times$ over vectorised FastJet (median $64\times$), with 100 % jet-count agreement everywhere. Benchmarked on an **NVIDIA V100-class** GPU.

$R = 1.0$ rows shown. Against the per-jet FastJet interface the gap is roughly *two orders of magnitude*.

# Backup — where the time goes

**Nsight Systems** — share of GPU time

| kernel | % GPU | avg/call |
|---|---|---|
| `_cluster_large_kernel` | **97.6** | 5.47 ms |
| `_decode_kernel` | 1.5 | 82 $\mu$s |
| torch glue | $< 1$ | — |

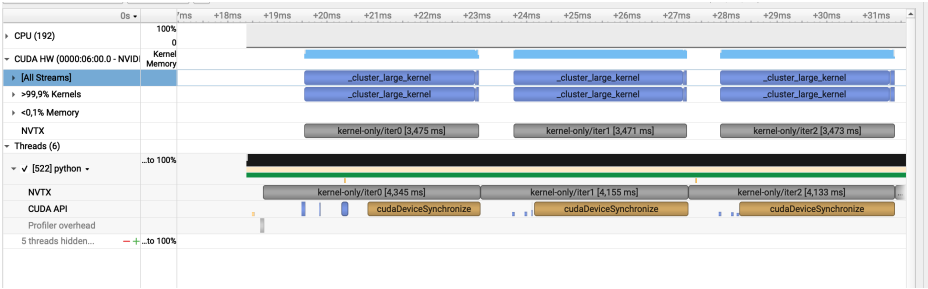
**No hidden overhead** — essentially all the time is real clustering work, and the memory row of the timeline is empty.

**Nsight Compute** — hardware counters

| metric | value |
|---|---|
| DRAM throughput | **0.02 %** |
| Compute (SM) throughput | 15.4 % |
| Registers / thread | 168 |
| Theoretical occupancy | 18.75 % |
| Achieved occupancy | **6.25 %** |
| Waves per SM | **0.32** |

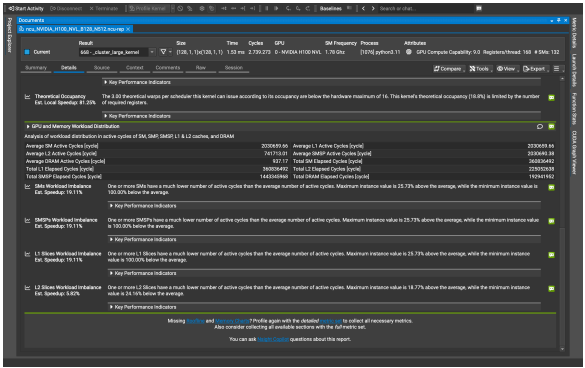
Not memory- or compute-bound — the GPU is simply **not filled**.

# Backup — the GPU timeline




Nsight Systems, three consecutive iterations. Each block is one clustering call; the tool labels the rows **“>99.9 % Kernels”** and **“<0.1 % Memory”**. The kernels run back to back with nothing in between and the memory row is empty — no host/device shuffling while clustering runs. That is what makes the speedup real rather than an artefact of where the timer was started.

# Backup — the profiler names the bottleneck




**The tool's own speedup estimates:**

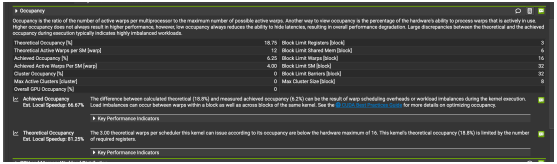
| finding | est. speedup |
|---|---|
| Theoretical occupancy (registers) | **81 %** |
| Achieved occupancy | 67 % |
| SM workload imbalance | 19 % |

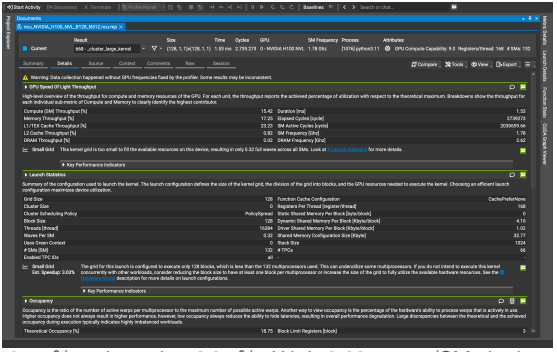
*"This kernel grid is too small to fill the available resources on this device, resulting in only 0.32 full waves across all SMs."*

Grid size **128 blocks** on a **132-SM** device — fewer blocks than the GPU has processors.

$\sim$**1.7–1.8**$\times$ more available from launch configuration and register budget — **not** from the algorithm.

# Backup — occupancy detail




Register pressure (168/thread) caps theoretical occupancy at 18.75 %; achieved is 6.25 %. With 0.32 waves/SM the kernel does not fill the GPU even once, while DRAM throughput is 0.02 % — this is parallelism starvation, not a bandwidth problem. Both are configuration inherited from smaller GPUs, not properties of the algorithm.

# Backup — the datasets

All benchmark data is **ATLAS Open Data** (CC0), read over the public redirector — no grid certificate required.

| dataset | used for |
|---|---|
| ATLAS Top Tagging Open Data | timing + FastJet agreement |
| 2020 ATLAS Jet Reconstruction | closure vs the experiment's own jets |

Constituents are calorimeter clusters, massless. Physics closures use toy showers compared against leading-log analytic predictions, so the prediction being tested is unambiguous.